

Parallel NLP Processing of Historical PDF Collections: A Case Study Comparing Multiprocessing and Dask in Academic Text Mining

1st Nicola Defonte

Postgraduate Program in Applied Computing (PPCA)
University of Brasília (UnB)
Brasília, Brazil
nico.defonte@gmail.com

2nd Claudio Henrique M. de Oliveira

Research Assistant, Department of Computer Science
University of Brasília (UnB)
Brasília, Brazil
claudio.oliveira@aluno.unb.br

3rd Marcelo Ladeira

Professor, Department of Computer Science
University of Brasília (UnB)
Brasília, Brazil
mladeira@unb.br

4th Gustavo van Erven

Professor, Department of Computer Science
University of Brasília (UnB)
Brasília, Brazil
gvanerven@gmail.com

Abstract—This paper presents the parallelization of an NLP pipeline developed for the Oxossi project, aimed at extracting inductive themes from large-scale collections of Portuguese-language academic PDFs. Two parallelization strategies are compared: Python’s built-in `multiprocessing` and the Dask task scheduler. Performance is evaluated using real-world corpora with variable file sizes. Results demonstrate that multiprocessing achieves higher efficiency for file-level processing, while Dask shows potential for future intra-document parallelization.

Index Terms—Parallel NLP, Text Mining, Multiprocessing, Dask, CRISP-DM, Historical Archives, PDF Processing

INTRODUCTION

The digitization of academic and historical archives has enabled large-scale computational methods for processing unstructured textual data. Within this context, the Oxossi project—developed by the postgraduate program in History at the University of Brasília (UnB)—aims to build a thematic search engine capable of extracting inductive themes and keywords from Portuguese-language academic documents, such as theses, dissertations, and articles.

This task lies at the intersection of Digital Humanities and Natural Language Processing (NLP). As the field evolves from digitization to computational analysis, new methodologies and tools are being explored to support research in the humanities [1], [2]. However, processing large PDF-based corpora poses specific challenges: documents may include complex layouts, scanned images, or noisy formatting, all of which complicate automatic text extraction and analysis.

To address these challenges, modern systems are adopting parallel parsing strategies to scale across CPU cores or distributed environments. Adaptive engines like AdaParse have demonstrated significant performance gains by dynamically allocating resources for complex documents [3]. Similarly,

MapReduce [4] introduced a foundational model for parallel data processing, now reflected in contemporary task-scheduling frameworks like Apache Spark and Dask.

In this study, we focus on accelerating a real-world academic NLP pipeline using two Python-native parallelization strategies: the built-in `multiprocessing` module and the Dask framework. The pipeline was developed for Oxossi to extract thematic summaries from noisy, varied academic PDFs. Our evaluation compares execution time, resource utilization, and scalability of these strategies when applied to a stratified benchmark dataset representative of the full corpus.

I. RELATED WORK

The CRISP-DM (Cross-Industry Standard Process for Data Mining) model [10] remains one of the most widely adopted methodologies for structuring data-driven projects. It provides an iterative framework that has been adapted to various NLP contexts [11], [12]. In historical document processing, hybrid pipelines often combine PDF parsing, linguistic normalization, and semantic network modeling. For instance, Lagunes and Hernández [13] used a CRISP-DM approach to process Latin American archives, integrating PyPDF2 and spaCy for textual normalization and thematic clustering.

Semantic keyword extraction methods such as TextRank [14] apply graph-based centrality to derive representative terms, with co-occurrence graphs commonly used for unsupervised text understanding. These approaches align with our inductive theme extraction model built on noun-pair proximity and PageRank scoring.

Parallelization plays a crucial role in handling large-scale document processing. Python’s `multiprocessing` module offers an accessible API for CPU-bound parallelism and has shown performance benefits in document extraction pipelines.

Silva and Fernandes [8] demonstrated that PDF text extraction can be substantially accelerated through multiprocessing, achieving speedups of up to 6× in real-world text mining tasks.

On the other hand, Dask is a task-based parallel computing framework designed for dynamic scheduling and scalable data workflows. Rocklin [5] introduced Dask as a lightweight alternative to Spark, particularly well-suited for blocked array and dataframe computations. Dask’s flexible task graph architecture allows it to scale from single-core machines to distributed clusters. Performance comparisons between Dask and Spark [6], [7] show that Dask performs competitively on memory-bound, document-centric workloads.

While most existing Dask studies focus on scientific or tabular data, its application to unstructured academic corpora remains underexplored. Our study provides empirical comparisons for both multiprocessing and Dask in a real-world historical NLP pipeline, identifying trade-offs in resource usage, scheduling overhead, and execution time.

II. METHODOLOGY

A. Problem Context

The sequential processing of approximately 9365 PDF documents presents a significant computational bottleneck. The main objective of this study is to parallelize the document processing pipeline in order to reduce execution time while maintaining semantic consistency and robustness in thematic extraction.

B. Data Characteristics

The dataset consists of approximately 9365 academic documents in PDF format, with file sizes ranging from 5 KB to 696 MB. Many files present irregular structures, including scanned images and poorly formatted text, requiring flexible text extraction techniques.

C. Computational Challenges and Objectives

Key challenges include handling heterogeneous file sizes, unstructured document layouts, and computational resource constraints. The goals of the pipeline are:

- Automatically extract dominant themes from academic texts;
- Preserve semantic consistency of extracted content;
- Handle unstructured and noisy data;
- Achieve significant reduction in total execution time through parallelization.

D. CRISP-DM and Semantic Graph-Based Text Mining

The project adopts the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology as a conceptual framework to structure its text mining pipeline. The NLP workflow integrates linguistic preprocessing, frequency analysis, n-gram detection, and semantic graph modeling.

Text is initially extracted from PDFs and processed using the `spaCy` NLP library. After preprocessing — including lemmatization, stopword removal, and token normalization — candidate keywords and bigrams are identified to simulate

a Bag-of-Words (BoW) representation enriched with n-gram features.

A co-occurrence graph is constructed by linking noun phrases based on proximity and frequency. The resulting semantic network is analyzed using PageRank centrality to identify the most representative keywords for each document.

E. Pipeline Architecture

The complete pipeline consists of the following stages:

- 1) **Text extraction:** PDF parsing and extraction of structured text;
- 2) **Preprocessing:** Cleaning, normalization, and lemmatization using `spaCy`;
- 3) **Theme detection:** Graph-based co-occurrence modeling and keyword extraction;
- 4) **Graph construction:** Semantic network creation with `NetworkX`.

F. Pipeline Architecture

The complete processing pipeline includes:

- 1) PDF extraction (`PyMuPDF` / `PyPDF2`)
- 2) File validation (empty text, corrupted files)
- 3) Language detection (`Lingua`)
- 4) Text cleaning (custom preprocessing)
- 5) Bigram extraction (`BitokenFinder`)
- 6) Lemmatization (`spaCy pt_core_news_sm`)
- 7) Co-occurrence graph creation (noun pairs)
- 8) PageRank computation (`NetworkX`)
- 9) Top theme extraction (Portuguese-filtered)

G. Evaluation Criteria

The extracted themes are evaluated based on their semantic relevance, diversity, and coherence with the document contents. Although no formal ground-truth dataset was available, qualitative validation was performed by comparing the extracted keywords to the document’s content as interpreted by human readers of Oxossi research team. This manual inspection confirmed that the top-ranked keywords generally reflected the dominant themes of each file. Future work may explore quantitative evaluation with labeled datasets or expert-based annotation. Extremely short or low-quality documents are automatically filtered out during preprocessing. Performance evaluation focuses primarily on processing time, CPU utilization, and memory consumption under both sequential and parallel execution.

H. Parallel Performance Evaluation

To evaluate parallel processing efficiency, the metric *Speedup* is defined as:

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}. \quad (1)$$

The parallel *Efficiency* (E) is then calculated as:

$$E = \frac{\text{Speedup}}{\text{Number of Workers}} \times 100\%. \quad (2)$$

This formula allows for comparison of resource utilization relative to the number of active workers.

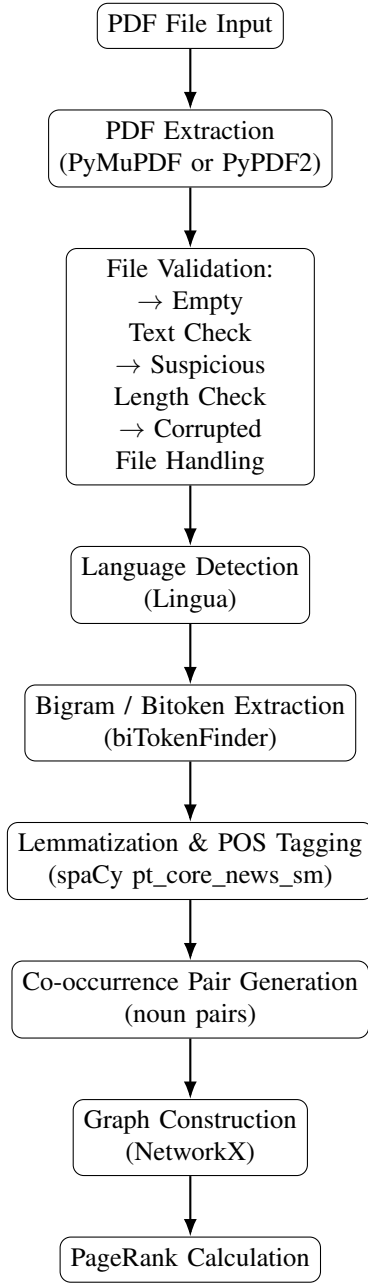


Fig. 1. Oxossi NLP Processing Pipeline (TikZ version).

I. Author Contributions

The NLP preprocessing module was originally developed by the Oxossi research team. Its adaptation and integration into the parallel processing framework presented in this study were performed by Nicola Defonte.

The complete parallel processing architecture—including PDF pipeline design, multiprocessing and Dask implementation, and performance benchmarking—was designed and implemented by Nicola Defonte as part of his master’s research.

Professors Marcelo Ladeira and Gustavo van Erven supervised the work and contributed to the revision of the experimental methodology and manuscript. Claudio Henrique

Marques de Oliveira assisted with testing and debugging of the Dask-based implementation.

All authors reviewed the results and approved the final manuscript.

III. EXPERIMENTAL SETUP

A. Hardware and Software Environment

All experiments were executed on a virtual private server (VPS) with 8 logical CPU cores, 32 GB of RAM, and 80 GB of storage. The environment used Python 3.8 and relied on libraries including spaCy, PyMuPDF, NetworkX, Lingua, and Dask.

B. Dataset Characteristics

The full Oxossi corpus contains 9 365 PDF documents. Table I summarizes file counts, size statistics, and the stratified size-bin distribution projected from our 500-file benchmark sample.

TABLE I
DATASET CHARACTERISTICS (9 365 PDFs)

Total documents	9 365
Average size (KB)	5 033.03
Minimum size (KB)	7.64
Maximum size (KB)	697 699.06
Total corpus size	52.5 GB
Size Distribution	
0–500 KB	1 354
500 KB–1 MB	1 277
1–2 MB	2 128
2–5 MB	2 478
≥5 MB	2 128

C. Sampling Methodology

To ensure our benchmarks reflect the diversity of the full corpus, we performed stratified random sampling along the primary axis of variability—file size. Documents were first grouped into five bins:

- 0–0.5 MB
- 0.5–1 MB
- 1–2 MB
- 2–5 MB
- ≥5 MB

Within each bin, files were selected at random without replacement using a fixed seed for reproducibility.

a) *Pilot sample (86 files)*: A “smoke test” comprising 0.9 % of the corpus: we randomly drew 86 files (17 per bin), covering small, medium, large, text-only, mixed, and scanned-only PDFs. This pilot runs in under 15 minutes on a single worker, allowing rapid validation of error handling, logging, and pipeline correctness.

b) *Full evaluation sample (500 files)*: For detailed performance metrics, we sampled 500 files (5.3 % of the corpus), maintaining the same proportional representation across the five size bins. At a 95 % confidence level, this sample size yields an estimated margin of error below 5 % for mean processing times, while completing in under one hour per framework.

D. Pilot and Full-Sample Statistics

TABLE II
SAMPLE STATISTICS (500 INPUT PDFs)

Metric	Value
Total input files	500
File Size (KB)	
Average	5 741.9
Minimum	13.4
Maximum	172 022.5
Total sample size (MB)	2 713.95
Size Distribution	
0–500 KB	70
500 KB–1 MB	66
1–2 MB	110
2–5 MB	128
≥5 MB	110

E. Parallelization Approaches

Two parallelization strategies were implemented:

The Oxossi project operates in a single-node environment with limited IT infrastructure and a team with minimal programming background. Therefore, Python’s built-in multiprocessing module and Dask were selected for their ease of integration, low setup overhead, and compatibility with CPU-bound workflows. More complex distributed frameworks such as Apache Spark were not considered due to infrastructure and expertise limitations.

1) *Multiprocessing (Python built-in module)*: The Python multiprocessing.Pool was employed to distribute PDF files across CPU cores. Each file was processed independently. A total of 7 workers were used, reserving one logical core for system processes. Benefits include:

- Native Python implementation with no external dependencies;
- Simple parallelism for independent tasks;
- Lower memory footprint.

2) *Dask (Task-Scheduling Framework)*: The Dask framework was introduced for more advanced parallelism. Dask builds directed acyclic graphs (DAGs) to schedule tasks, enabling more granular task management and potential intra-document parallelization. Benefits include:

- Dynamic scheduling of complex workflows;
- Compatibility with NumPy and pandas;
- Scalability to clusters or single-machine environments;
- Built-in dashboard for real-time resource monitoring.

F. Preliminary Testing and Parallel Performance

An initial evaluation was conducted using a sample of 86 PDF files, with sizes ranging from 5 KB to 40 MB, to benchmark both multiprocessing and Dask frameworks under controlled conditions.

Using Python’s multiprocessing module with 7 workers, total processing time was reduced from 54 minutes 43 seconds (3283.46 s) in sequential execution to 10 minutes 46 seconds (646.22 s) in parallel execution. This resulted in a speedup of 5.08× and a parallel efficiency of 72%.

For Dask, parallel execution time was measured at 11 minutes 47 seconds (706.87 s), corresponding to a speedup of 4.64× and a parallel efficiency of 66.36%. Although Dask delivered reasonable performance in the small-scale test, its speedup remained slightly lower than multiprocessing under identical hardware and workload conditions.

In both experiments, 5 files were automatically skipped due to extraction failures. These failures were caused primarily by:

- **Empty text extraction**: Indicating likely image-based PDFs with no embedded text.
- **File integrity errors**: Resulting from corrupted or unreadable PDF structures.

These preliminary tests provided useful insight into the practical limitations and scheduling overheads of each parallelization strategy even before scaling to the full dataset.

G. Full Evaluation

Subsequent experiments processed 500 files under both parallelization frameworks:

1) Sequential Baseline (1 Worker):

- Total files: 500;
- Files processed: 484;
- Total time: 3h 58m 34s (14313.75 seconds);
- Average processing time per file: 29.57 seconds.

2) Multiprocessing (7 Workers):

- Total time: 43m 38s (2617.91 seconds);
- Speedup: 5.47×;
- Efficiency: 78.14%;
- Average CPU usage: 95.7%;
- Average memory usage: 69.9%.

3) Dask (7 Workers):

- Total time: 67m 38s (4058.25 seconds);
- Speedup: 3.53×;
- Efficiency: 50.39%;
- Average CPU usage: 96.5%;
- Average memory usage: 92.5%.

IV. RESULTS AND DISCUSSION

The parallelization experiments were extended to a larger dataset of 500 academic PDF files, with heterogeneous file sizes ranging from a few kilobytes to over 170 MB. The goal was to evaluate execution time, resource utilization, and scalability across both parallelization strategies.

A. Sequential Baseline

The full sequential execution (single worker) processed 484 out of 500 files successfully, with 16 files skipped due to extraction failures. The total execution time was 3 hours 58 minutes 34 seconds (14313.75 s), yielding an average processing time per file of 29.57 s.

B. Multiprocessing Performance

Using 7 workers, the multiprocessing implementation reduced total processing time to 43 minutes 38 seconds (2617.91 s), achieving a speedup of 5.47× and parallel efficiency of 78.14%. CPU utilization averaged 95.7% while memory usage remained moderate at approximately 69.9%.

C. Dask Performance

The Dask implementation yielded a total processing time of 67 minutes 38 seconds (4058.25 s), corresponding to a speedup of 3.53× and parallel efficiency of 50.39%. While CPU utilization remained high (96.5%), memory usage increased significantly, averaging 92.5% and peaking at 99.3%.

D. Discussion

Overall, multiprocessing consistently outperformed Dask in both the preliminary and full-scale evaluations, particularly for file-level parallelism on heterogeneous and relatively small PDF documents. Dask incurred higher scheduling overhead and memory consumption, which reduced its scalability on single-node CPU workloads.

However, Dask remains promising for future work involving intra-file parallelization, dynamic task graphs, or larger distributed environments. Its flexible architecture and task scheduler may provide advantages in scenarios with finer-grained parallel workloads, larger datasets, or mixed computational stages.

TABLE III
EXECUTION PERFORMANCE SUMMARY (500 FILES)

Method	Cores	Total Time (s)	Speedup	Efficiency
Sequential	1	14313.75	1.00×	100%
Multiprocessing	7	2617.91	5.47×	78.14%
Dask	7	4058.25	3.53×	50.39%

TABLE IV
RESOURCE CONSUMPTION SUMMARY (500 FILES)

Method	Cores	Average CPU (%)	Average RAM (%)
Sequential	1	42.7%	47.3%
Multiprocessing	7	95.7%	69.9%
Dask	7	96.5%	92.5%

V. CONCLUSION AND FUTURE WORK

This study presented a comparative evaluation of two parallelization frameworks—Python’s multiprocessing module and the Dask task scheduler—for accelerating an NLP pipeline designed to extract inductive themes from large collections of historical and academic PDF documents.

The results demonstrate that multiprocessing achieved superior performance on the tested file-based workloads. For the full dataset of 500 PDFs, multiprocessing achieved a speedup of 5.47× and parallel efficiency of 78.14%, while Dask yielded a speedup of 3.53× and efficiency of 50.39%. Multiprocessing consistently required less memory overhead while fully utilizing available CPU cores. Dask exhibited higher scheduling overhead and significantly higher memory consumption, limiting its efficiency in single-node, file-level parallelism scenarios.

Nevertheless, Dask remains a promising alternative for future stages of this project. Its task graph model, scalability, and cluster support may offer advantages for more complex

pipelines that involve intra-document parallelism or distributed data processing. Future work will explore Dask’s advanced capabilities, integration with GPU acceleration, and optimization of memory management for processing larger datasets.

The proposed architecture demonstrates the feasibility of applying parallel NLP techniques to unstructured historical corpora while preserving semantic consistency and extraction quality, contributing to the growing field of computational humanities.

A. Limitations

The current study focuses on file-level parallelism for CPU-bound workloads on single-node systems. The results may vary for highly scanned documents, poorly OCR-processed PDFs, or environments where distributed or GPU acceleration is available. These scenarios are planned for future investigations.

REFERENCES

- [1] S. Castano, A. Ferrara, S. Montanelli, and F. Periti, “From Digital to Computational Humanities: The VAST Project Vision,” in *CEUR Workshop Proceedings*, 2021.
- [2] L. Manovich, *Understanding Digital Humanities*. London, U.K.: Palgrave Macmillan, 2012.
- [3] C. Siebensschuh, K. Hippe, O. Gokdemir, *et al.*, “AdaParse: An Adaptive Parallel PDF Parsing and Resource Scaling Engine,” *arXiv preprint arXiv:2505.01435*, 2025.
- [4] J. Dean and S. Ghemawat, “MapReduce: Simplified data processing on large clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [5] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proc. of the 14th Python in Science Conference (SciPy)*, 2015, pp. 130–136. DOI: 10.25080/Majora-7b98e3ed-013.
- [6] M. Dugré, V. Hayot-Sasson, and T. Glatard, “A performance comparison of Dask and Apache Spark for data-intensive neuroimaging pipelines,” *arXiv preprint arXiv:1907.13030*, 2019.
- [7] M. Dugré, V. Hayot-Sasson, and T. Glatard, “Performance comparison of Dask and Apache Spark on HPC systems for Neuroimaging,” *arXiv preprint arXiv:2406.01409*, 2024.
- [8] R. F. Silva and L. M. Fernandes, “Accelerating text extraction from PDFs with Python multiprocessing,” *Journal of Document Analysis and Recognition*, vol. 23, no. 2, pp. 147–159, 2020.
- [9] S. M. Alzahrani, “Parallel programming approaches for efficient natural language processing of big data,” *Int. J. Comput. Sci. Inf. Secur.*, vol. 14, no. 10, pp. 1–18, 2016.
- [10] R. Wirth and J. Hipp, “CRISP-DM: Towards a standard process model for data mining,” in *Proc. of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, 2000. [Online]. Available: <https://www.the-modeling-agency.com/crisp-dm.pdf>
- [11] ScitePress, “Applying CRISP-DM for Text Mining Projects,” in *Proc. of the 12th International Conference on Agents and Artificial Intelligence (ICAART)*, 2020.
- [12] B. Ayele, “CRISP-IM: An Extension of CRISP-DM for Idea Mining Using Topic Models,” *Journal of Big Data*, vol. 8, no. 1, 2021.
- [13] G. Lagunes and C. Hernández, “Text Mining of Latin American Historical Documents using CRISP-DM,” in *Proc. of the 6th International Conference on Data Science and Information Technology (DSIT)*, 2021.
- [14] R. Mihalcea and P. Tarau, “TextRank: Bringing Order into Texts,” in *Proc. of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2004.
- [15] Dask Development Team, “Dask: Parallel Computing with Task Scheduling,” 2024. [Online]. Available: <https://docs.dask.org/en/stable/>